Photo by congerdesign on pixabay

Lecture 2

–

Model Evaluation

Schwalbenschwanz (*Papilio machaon*)

- Der Schwalbenschwanz ist ein Schmetterling aus der Familie der Ritterfalter. Er ist einer der größten und auffälligsten Tagfalter des deutschsprachigen Raums und hat eine Flügelspannweite von 50 bis 75 Millimetern.
- Seine Flügel sind auffällig gemustert mit einem blauen Streifen am unteren Rand und den länglichen Fortsätzen.
- Er war Schmetterling des Jahres 2006.
- Die Bestände des beeindruckenden Falters haben sich in den letzten Jahren erholt, und so ist er wieder ab und an auf Blumenwiesen oder in Gärten zu sehen.

Quellen:
* https://de.wikipedia.org/wiki/Schwalbenschwanz_(Schmetterling)
* https://www.plantura.garden/gruenes-leben/schmetterlinge-die-10-beliebtesten-heimischen-schmetterlingsarten

Photo by Johnnys_pic on pixabay

Model Evaluation

1. Classification Scores
2. Bias-Variance Tradeoff
3. Score Estimation Methods
4. Hyperparameter Tuning

Exercises

Exercise 1

Homework Review – Classifier Scores and ROC Chart

Model Evaluation

1. Classification Scores
2. Bias-Variance Tradeoff
3. Score Estimation Methods
4. Hyperparameter Tuning

Bias - Underfitting

- Bias: degree to which a model's predictions differ from the true values.
- Source of error: from erroneous assumptions in the learning algorithm
- Underfitting: Models bias too high → cannot capture the underlying patterns in the data
- How to notice underfitting
  - "Poor" performance
  - What is "poor"? Depends on problem/dataset!
  - In practise: try different biases / models!

Variance - Overfitting

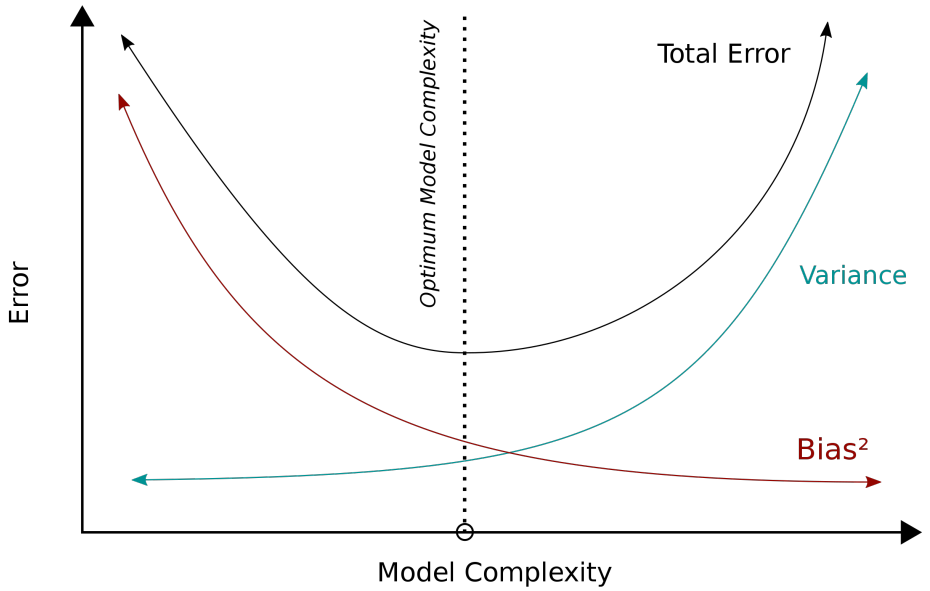
- Variance: degree to which a model's predictions are sensitive to small fluctuations in the training data.
- Source of error: from sensitivity to small fluctuations in the training set
- Overfitting: Variance so high that model is fitting the noise in the data and not the underlying patterns.
- How to notice overfitting
  - Split dataset into at least two chunks called training dataset and testing dataset (Holdout method)
  - Train model on training dataset only
  - Evaluate performance on both datasets
  - Performance is considerably worse on testing dataset than on training dataset → Overfitting

Bias-Variance Trade-Off

- Bias-Variance Trade-Off: central problem in supervised learning
  - More ‘complex’ model: bias decreases and variance increases.
  - More ‘simple’ model: bias increases and variance decreases.

- Goal: find optimal balance between bias and variance
  - model fits the data well without overfitting, i.e.
    do both: capture the regularities in the training data,
    but also generalize well to unseen data
    → Typically, very hard to do both simultaneously

- Techniques used to achieve the goal
  - Model-Inherent Complexity Management
  - Regularization
  - Ensemble learning



Bias-Variance Dilemma (Trade-off)

Low-Bias = High-Variance Models

- Complex – represent training data more accurately
- Also represent noise in training data → predictions less accurate (overfit) and unstable
- Examples: ID3, High-order polynomial regression

High-Bias = Low-Variance Models

- Tend to be simple
- May fail to capture important regularities in the data (underfit)
- Examples: Linear regression, decision stumps (=decision trees with only one split in the root)

Model Evaluation

1. Classification Scores
2. Bias-Variance Tradeoff
3. Score Estimation Methods
4. Hyperparameter Tuning

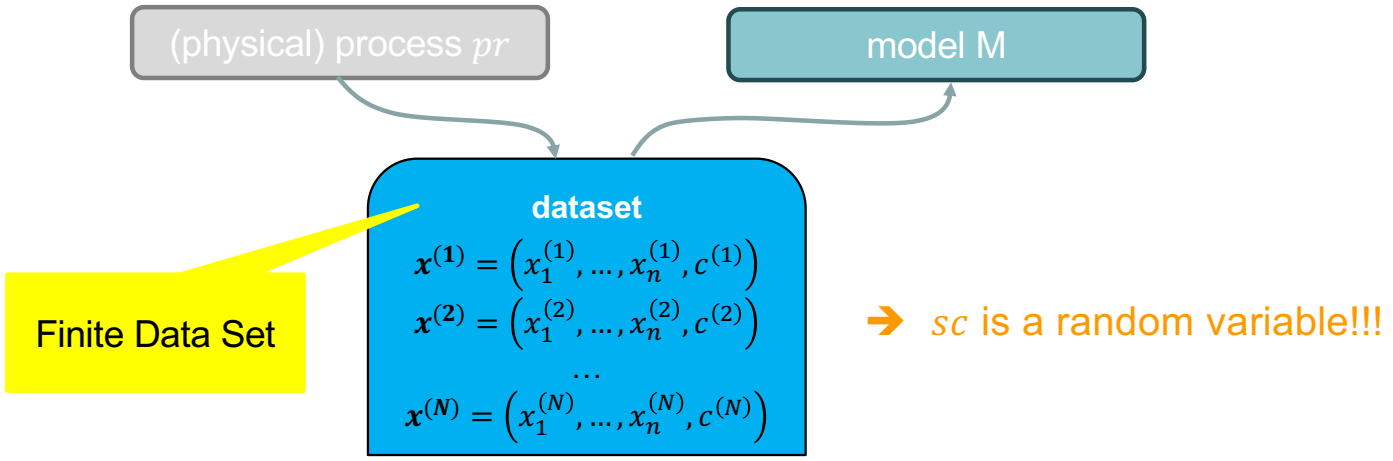
Motivation

Challenge:
1) Which is the best classification/regression algorithm for a given problem?
2) Which is the most suitable parameterization of an algorithm for a given problem?

➔ Select a score sc to measure the quality

➔ Calculate the true value of the score

But: Limited information



➔ sc is a random variable!!!

Score as a random variable

- Score sc is a random variable
- True value is unknown

➔ Calculate expected value of the score $\widehat{sc}$
based on the set of observations sampled from the process pr

➔ Choice of how to deal with the given dataset:
  - Resubstitution
  - Hold-Out
  - Cross-Validation and Leave-One-Out
  - Bootstrap

dataset

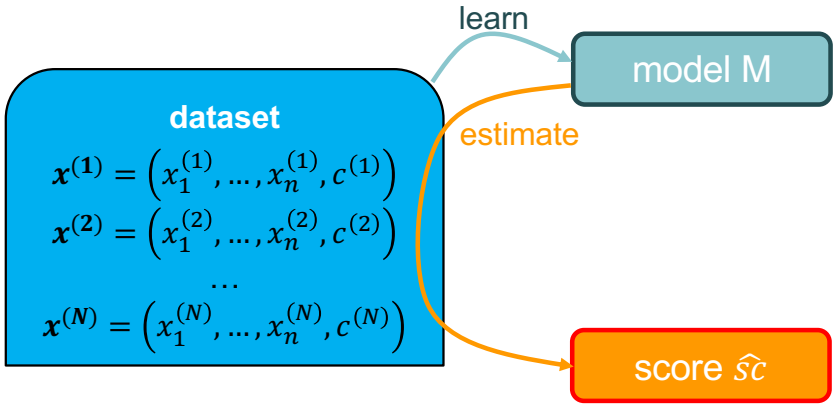
$$\boldsymbol{x^{(1)}} = \left(x_1^{(1)}, \ldots, x_n^{(1)}, c^{(1)}\right)$$
$$\boldsymbol{x^{(2)}} = \left(x_1^{(2)}, \ldots, x_n^{(2)}, c^{(2)}\right)$$
$$\ldots$$
$$\boldsymbol{x^{(N)}} = \left(x_1^{(N)}, \ldots, x_n^{(N)}, c^{(N)}\right)$$

Resubstitution

- Use full dataset for both training and score estimation
- Simplest estimation method
- Model has seen the data used for estimating the score

➔ Too optimistic (overfitting problem)

➔ Bad estimator of the true classification score !!!

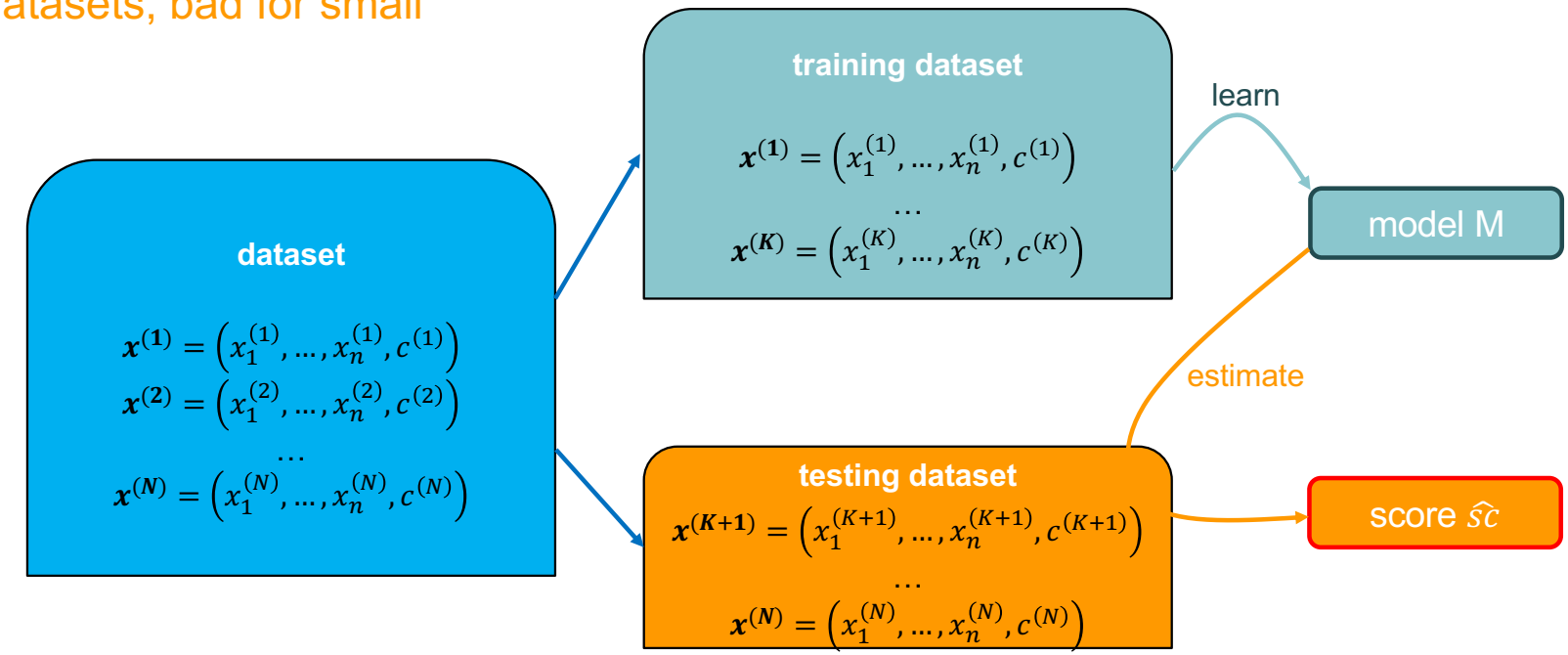


Hold-Out

- Split dataset into training and testing sets
- Often pessimistic estimation of true score

➔ Useful for large datasets, bad for small datasets

if similar value for both testing dataset and training data set, then model rather good



Score-Estimation via Hold-Out - Discussion

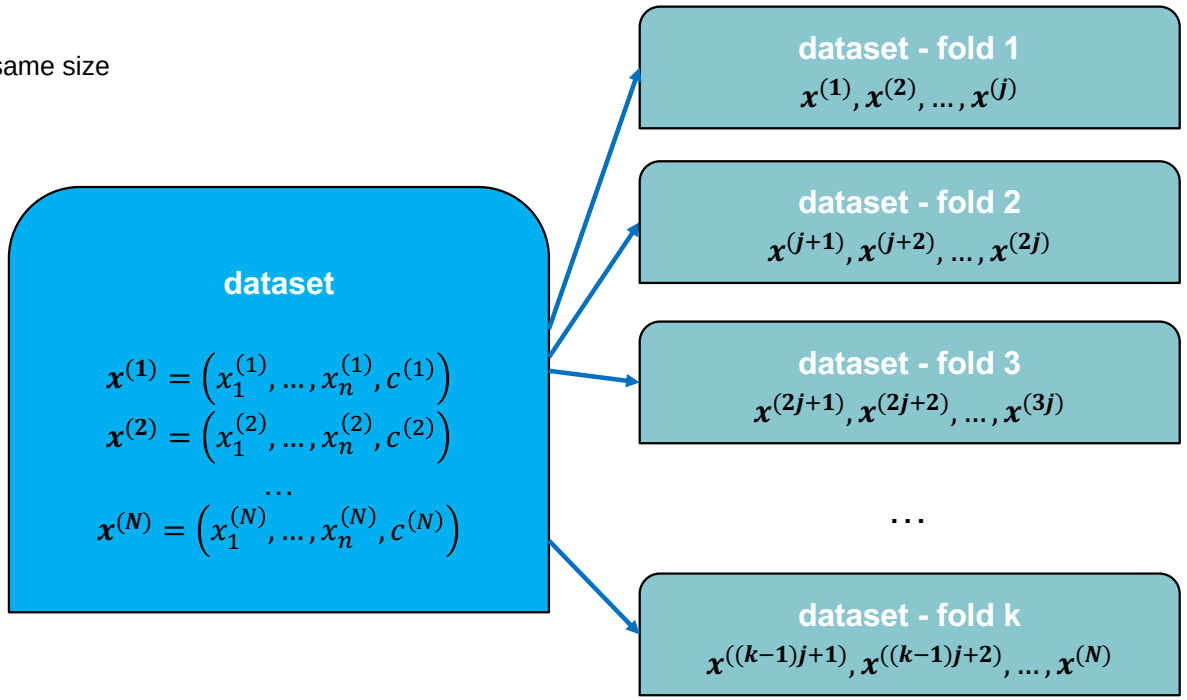
- Better estimate of the score than using full dataset for training and testing
- Can compare score on train and test data to recognize overfitting
  - Good on train, bad on test: overfitting
  - Good on both: model generalizes well
- Issues: validated model only once
  - No way to measure variance of the estimated score
    - The split may by chance have been very conducive to this model
    - The split may have introduced a large skew into the data
  - Significantly reduced size of the training dataset
- Summary

  ➔ Useful only for large datasets - but what is “large”?
- Code: `sklearn.model_selection.train_test_split(…)`

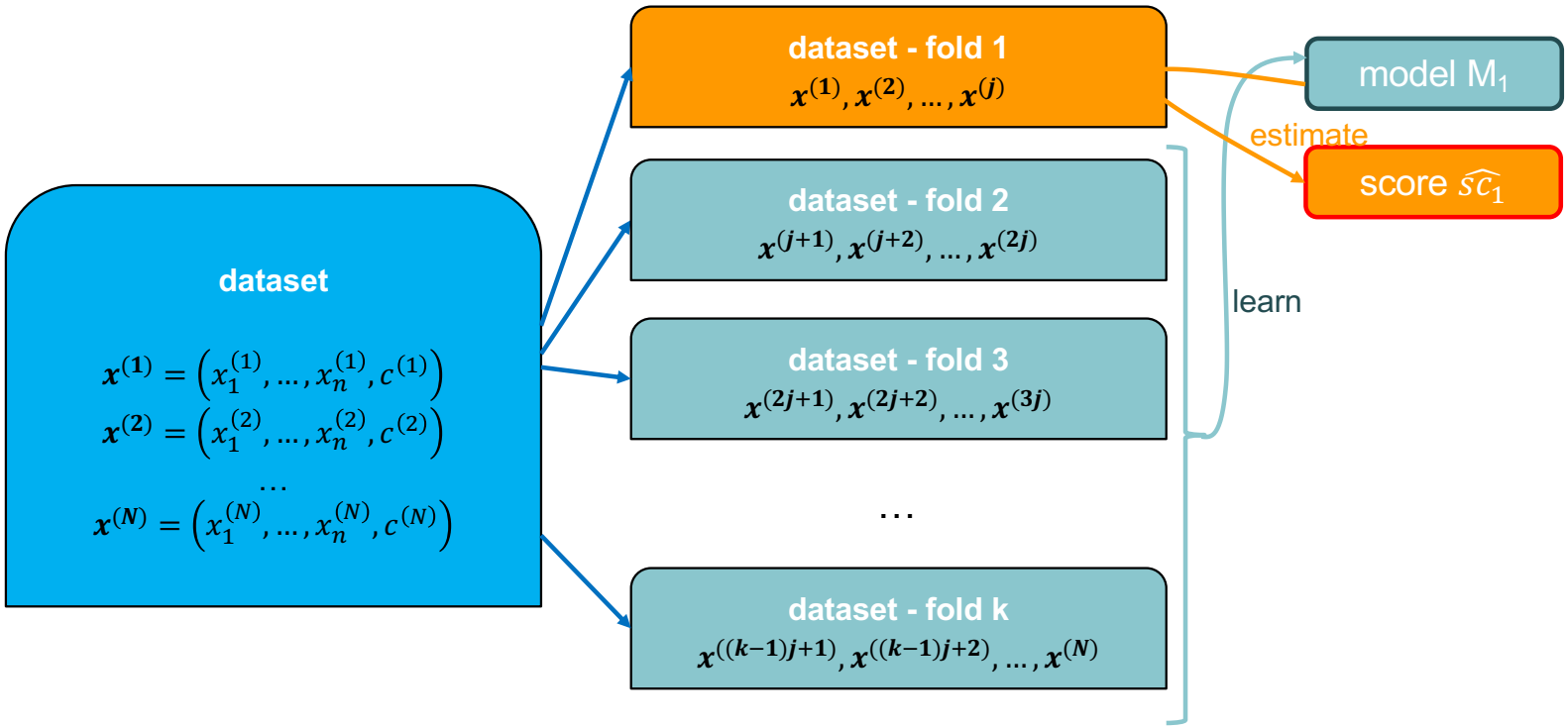
k-fold cross-validation (1)

Let $j = \lfloor N/k \rfloor$

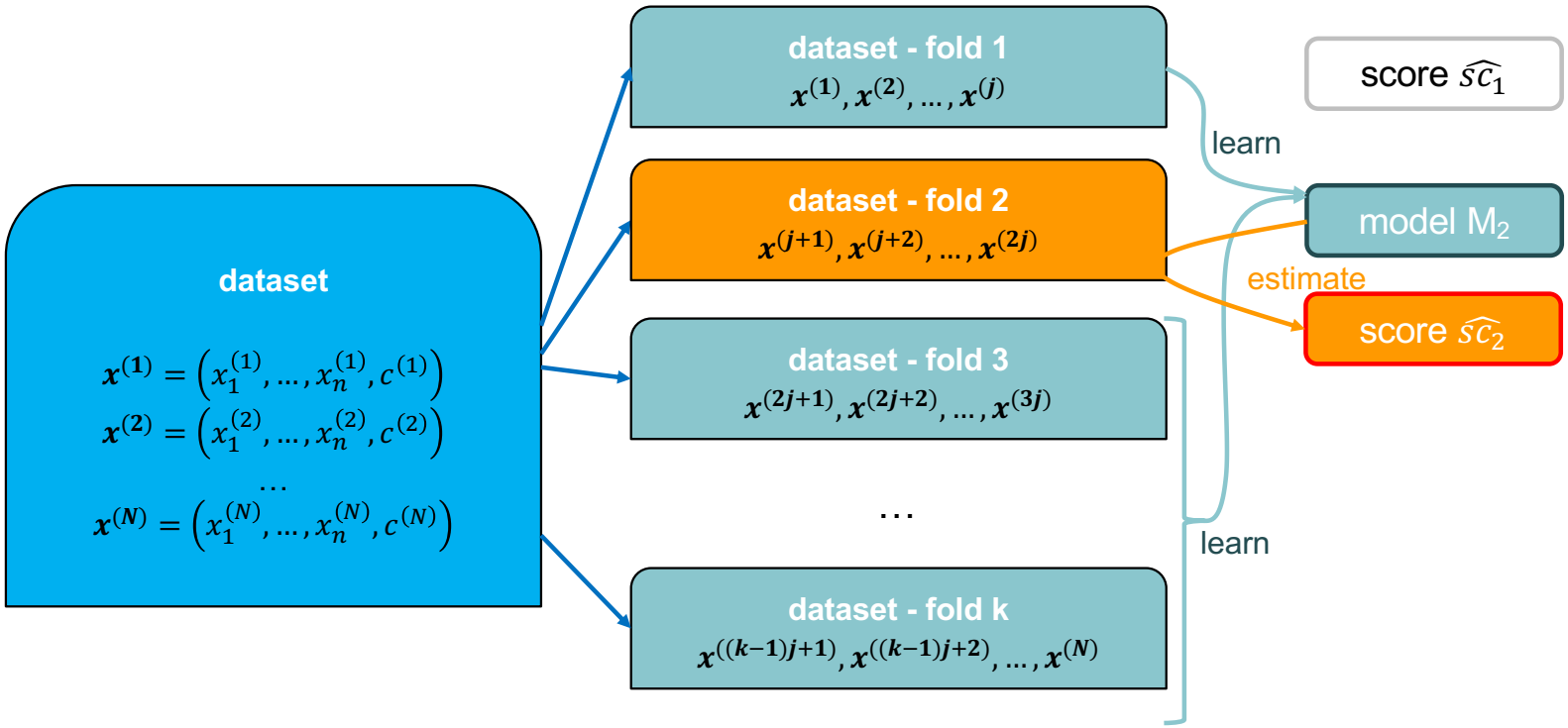
split data set into k groups of same size



k-fold cross-validation (2)



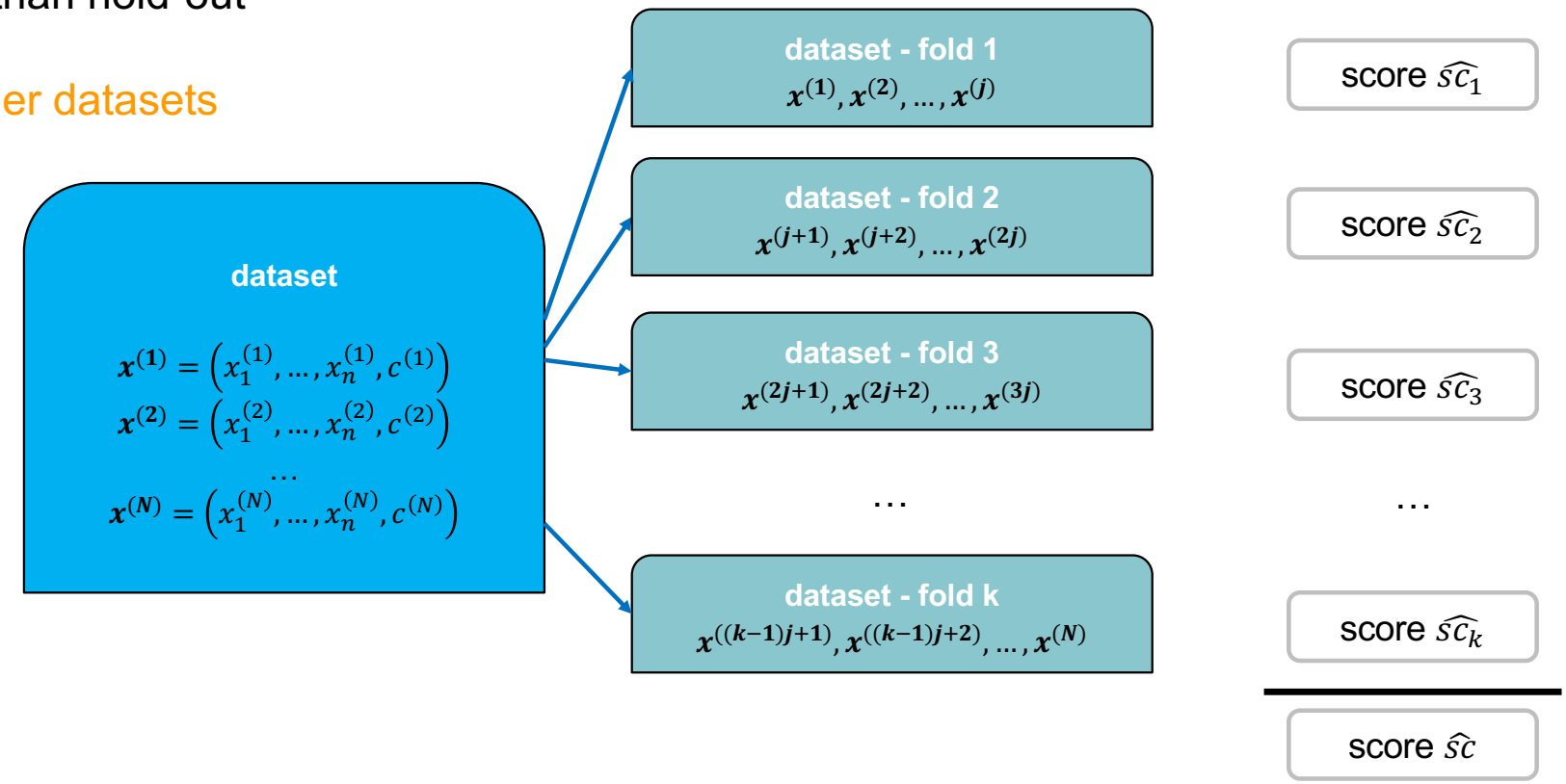
k-fold cross-validation (3)



k-fold cross-validation (4)

- Less pessimistic than hold-out

➔ Suitable for smaller datasets



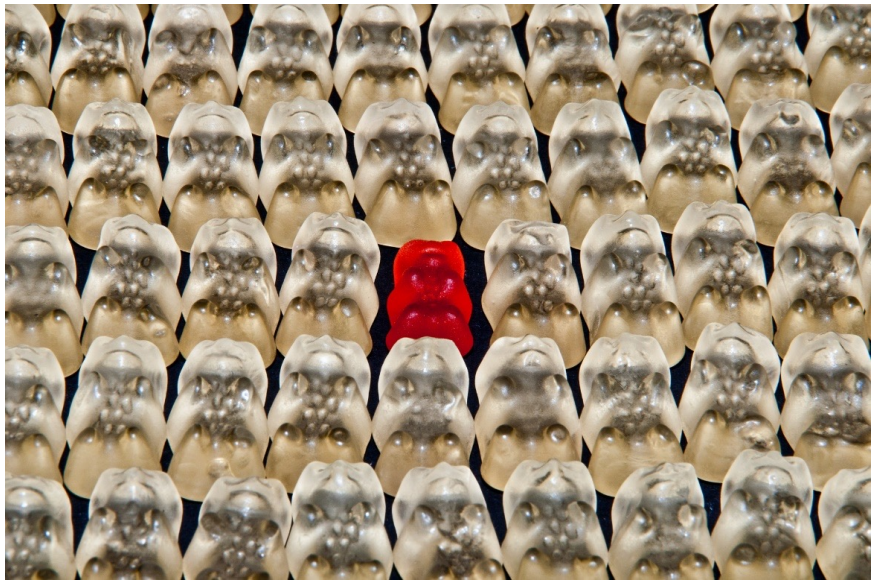
deployed model will always be trained on the whole data set, meaning its effectiveness cannot be judged via its score, as it is hard to get one (without extra data)

Score-Estimation via Cross-Validation - Discussion

- Validated model k-times
- Reduced size of training data, but less so than for hold-out
- Can compare (mean) score on train and test data to recognize overfitting
  - Good on train, bad on test: overfitting
  - Good on both: model generalizes well
- Can recognize high variance of the model by high variance of the (test) scores
- Challenge: k-times more expensive (training time) than hold-out
- Summary
  - ➔ Superior to Hold-Out
- Code: `sklearn.model_selection.cross_val_score(…)`

Leave-one-out cross-validation (5)

- Special case where $k = N$
  - Each fold contains only one observations
  - Calculate N scores each based on only one observation
- High computational cost
- Better estimation of true score, but less stable (=higher variance)

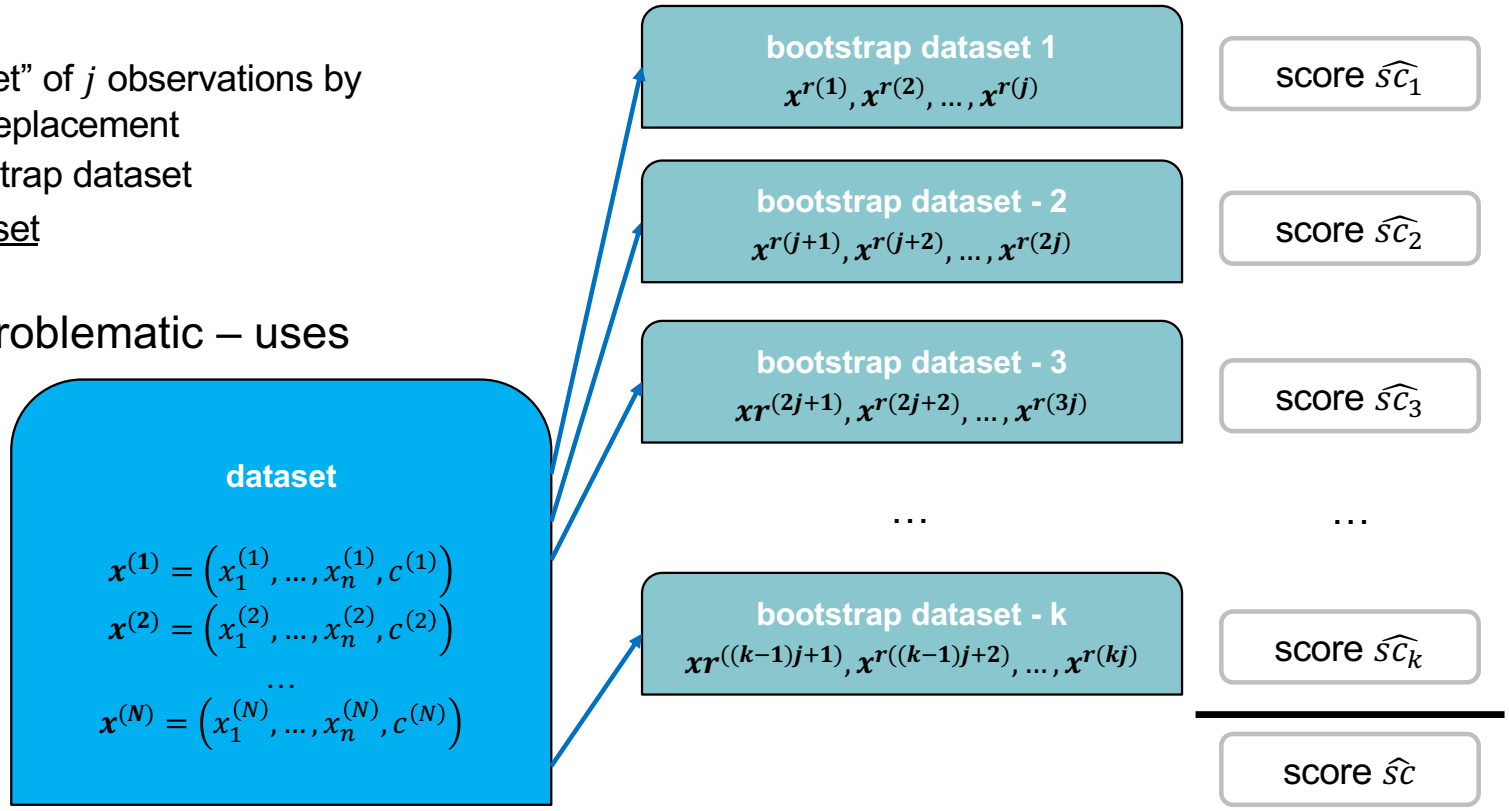


Bootstrap

- General Idea
- Repeat k times:
  - create "bootstrap dataset" of j observations by random sampling with replacement
  - Train classifier on bootstrap dataset
  - Estimate using full dataset

➔ Optimistic estimation problematic – uses data used for training!

➔ Not recommended



.632-Bootstrap

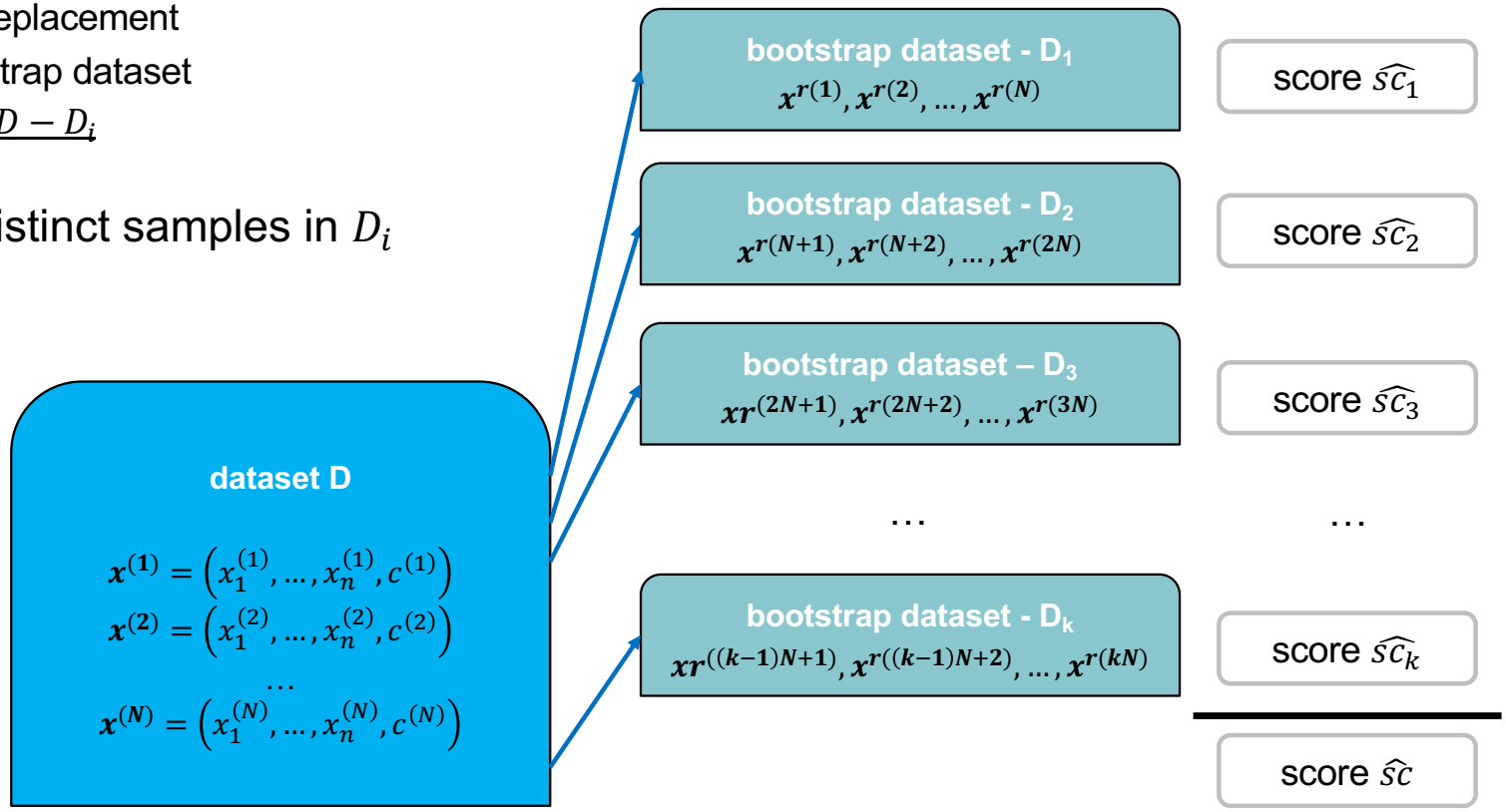
- Repeat k times:
  - create "bootstrap dataset" D_i of N observations by random sampling with replacement
  - Train classifier on bootstrap dataset
  - Estimate using dataset $D - D_i$

➔ Expected number of distinct samples in D_i is approx. $0.632 * N$

➔ No training data used for estimation

➔ Pessimistic estimate

➔ Useful for small datasets



Score Estimation Summary

- Resubstitution
  - Do not use
- Hold-Out
  - Good for large datasets
  - Computationally inexpensive
  - Frequent splits 80%/20% or 70%/30% (train/test)
  - Can be done repeatedly to improve estimation (repeated hold-out)
- Cross-Validation and Leave-One-Out
  - Good for small datasets
  - Computationally expensive
- Bootstrap
  - .632 Bootstrap good for small datasets
  - Computationally expensive

Let‘s do 4
short questions

Every question has
4 statements:
3 of which are correct,
1 of which is wrong.

<u>Pick the wrong one!</u>

Question 1

It is challenging to measure the quality *sc* of a classifier/regressor

A) Because *sc* is a random variable

B) Because there are many possible option for defining a good *sc*

C) Because we only have limited information on the process behind *sc*

D) Because we can only sample a limited number of values for *sc*

Question 2

Resubstitution

A) gives a very good estimate of the score on the training data

technically correct, bc it is correct for training data but we do not care for that data set

B) is easy to implement and therefore should be used for simple classifiers

C) is usually too optimistic

D) uses the full dataset for both training and score estimation

Question 3

Leave-one-out

A) is a special case of k-fold-cross validation

B) is quite unstable and the measurements vary considerably

C) is very useful for large datasets

D) is very expensive

Question 4

The .632-Bootstrap for N available observations

A) uses the same number of different observations for training and evaluation

B) does not use any training data for computing the classifier score

C) draws N samples for every bootstrap samples

D) is a pessimistic estimate but nevertheless useful for small datasets

Model Evaluation

1. Classification Scores
2. Bias-Variance Tradeoff
3. Score Estimation Methods
4. Hyperparameter Tuning

Hyperparameter - Tuning

- Hyperparameter Tuning: select optimal hyperparameter values for a machine learning model
  - Hyperparameters are set before training and not changed/learned during training
  - ”make or break” the model
  - “trial and error”: select hyperparameter values, train model, evaluate performance.
- Common Hyperparameter Tuning/Selection Methods
  - Grid search: specify a grid of hyperparameter values try all possible combinations of values.
  - Random search: randomly select hyperparameter values.
  - Bayesian optimization: use probabilistic model to predict performance for different hyperparameter values and select next set of hyperparameters to try based on predicted performance.
- Which data to use for evaluation of a set of Hyperparameter Values?
  - Cannot use training data – too optimistic
  - Cannot use testing data – must not be “tainted” (avoid leakage)
  - ➔ Need additional dataset, the validation data

Hyperparameter Tuning and Model Evaluation

- Need to estimate model score to tune hyperparameters
  - Pick the optimal hyperparameters based on the score
- Need to estimate model score to evaluate the model
  - Recognize Overfitting and evaluate model from the business perspective

➔ Need to evalute the score twice and independently

➔ Can chose (any) two different score estimation methods!

- Popular Choices
  - Hold-Out for both hyperparameter-tuning and model-evaluation: Train-Validation-Test Split (nested-hold-out) – simplest choice
  - Cross-validation for hyperparameter-tuning and Hold-Out for model-evaluation
  - Cross-validation for both hyperparameter-tuning and model-evaluation: nested cross-validation

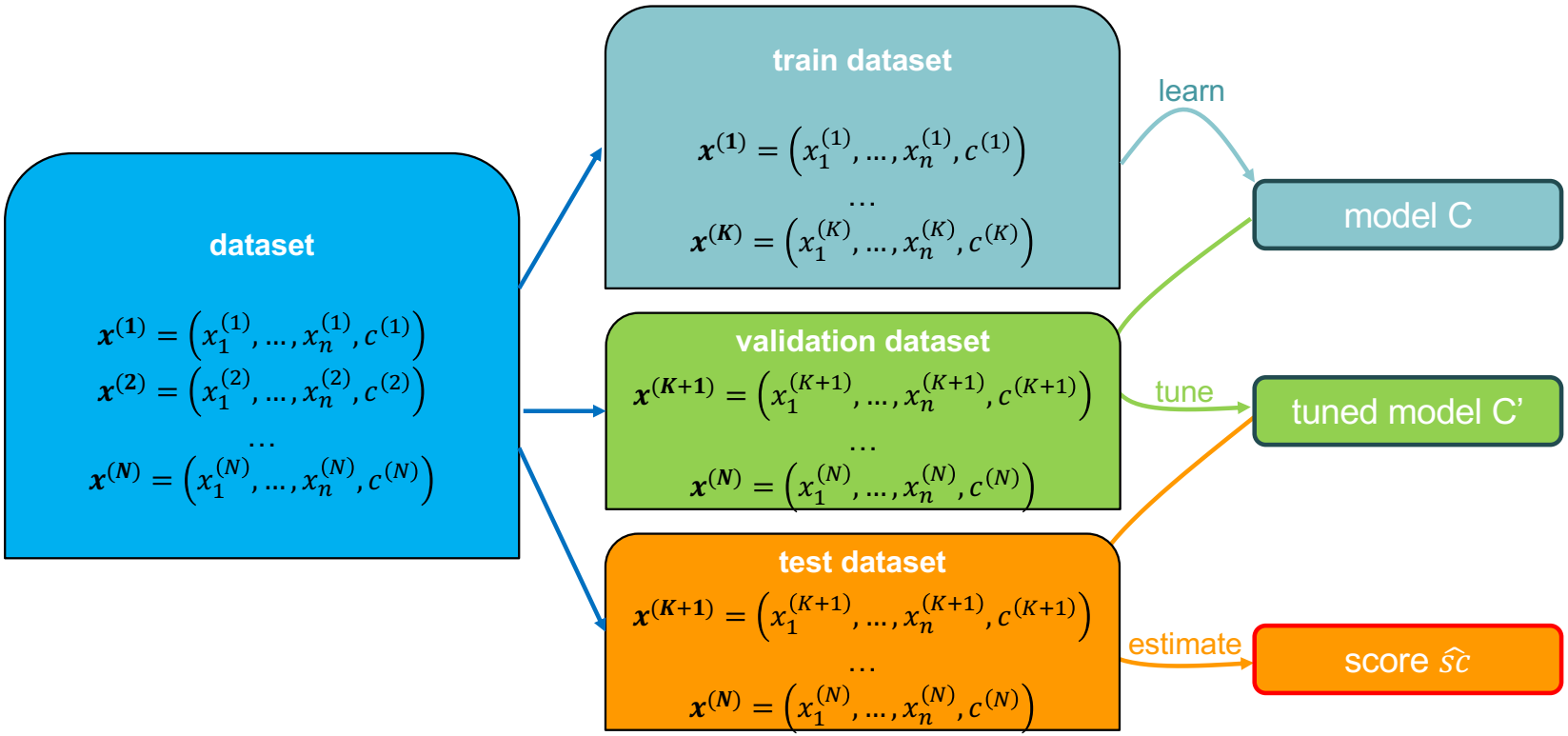
Hyperparameter Tuning & Score-Estimation via Train-Validation-Test Split

- Split dataset into train, validation and test datasets
  - Common splits: 60:20:20 or 50:25:25
- Tune Hyperparameters using the validation dataset (repeatedly)
  - Train Model on train dataset multiple times for different sets of hyperparameters
  - Evaluate score for each set of hyperparameters on validation set
  - Pick the best hyperparameters
- Compute final performance metric (score) on train+validation and test dataset (once only!)
  - Bad on both: underfitting
  - Good on train + validation, bad on test dataset: overfitting
  - Good on both: model generalizes well

+ 1 (final performace metric)

- Computational cost: one model trained per set of hyperparameter values

Hyperparameter Tuning & Score-Estimation via Train-Validation-Test Split - Visually



Hyperparameter Tuning & Score-Estimation via Train-Validation-Test Split – Discussion

- Advantages
  - Test dataset is never used to train/tune – no data leakage
  - Good performance
    - model trained once per set of hyperparameter value h: O(h)
  - Easy to implement (simple loop)

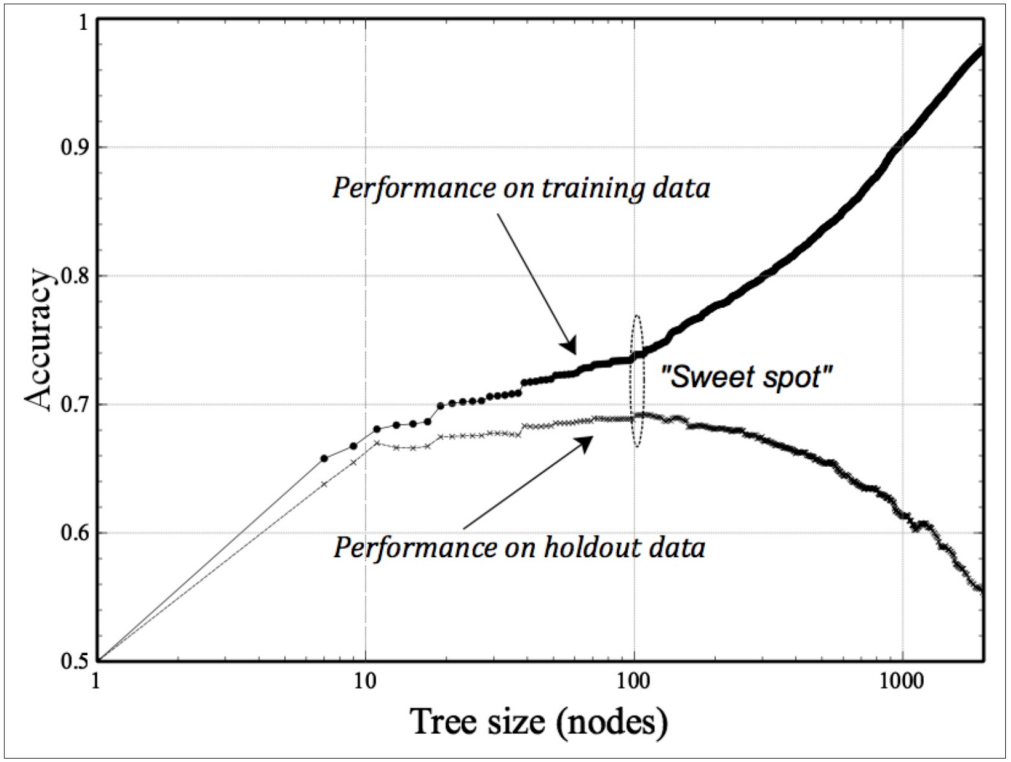
- Disadvantages – same as (simple) Hold-out (but twice)
  - Validated model only once per set of hyperparameter values
    - The split may by chance have been very conducive to this set of hyperparameter values
    - The split may have introduced a large skew into the data
  - Validated model only once to evaluate model
    - The split may by chance have been very conducive model
    - The split may have introduced a large skew into the data
  - Significantly reduced size of the training dataset

➔ Useful only for large datasets - but what is “large”?

- Code: use `sklearn.model_selection.train_test_split(…)` twice

Fitting Graph

- Fitting Graph: Plot performance metric over model complexity
  - Example: accuracy over leaf nodes of a Decision Tree
- Used to find the optimal model complexity / to tune hyperparameters visually

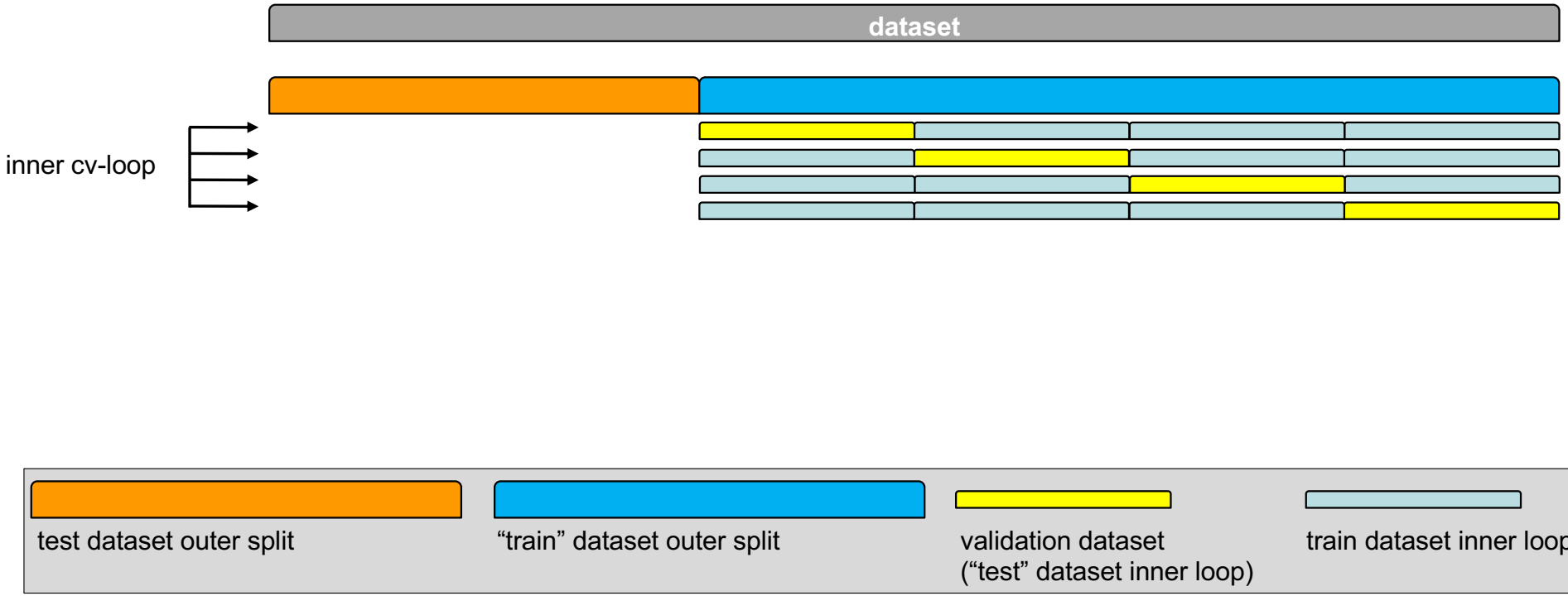


Source: Data Science for Business, p117

Cross-validation for hyperparameter-tuning and Hold-Out for model-evaluation

- Split dataset into (outer) “train” and test sets and apply k-fold cross-validation to “train”
  - Common splits: 80%-20% or 70%-30%
- Tune Hyperparameters using cross-validation on the outer “train” data
  - For each different set of hyperparameters: train Model k-times (on inner train-folds) and compute score (on inner test-fold). Compute the mean score.
  - Pick the best hyperparameters based on the mean score.
  - Evaluate variance of the best hyperparameter set
- Compute final performance metric (score) on (outer) train and (outer) test dataset (once only!)
  - Good on train + validation, bad on test dataset: overfitting
  - Good on both: model generalizes well

Cross-validation for hyperparameter-tuning and Hold-Out for model-evaluation – Visually for for k = 4



Cross-validation for hyperparameter-tuning and Hold-Out for model-evaluation – Discussion

- Advantages
  - Test dataset is never used to train/tune – no data leakage
    - Can compare (mean) score on train and test data to recognize overfitting
  - Reasonable performance and easy to implement
    - Model trained k-times per set of hyperparameter values: O(k*h)
  - Scored model k-times for each set of hyperparameter
    - Can recognize high variance of hyperparameter-value-set by high variance of the scores
- Disadvantages
  - Scored model only once to evaluate model
    - The split may by chance have been very conducive model
    - The split may have introduced a large skew into the data
  - Reduced size of training data, but less so than nested-hold-out

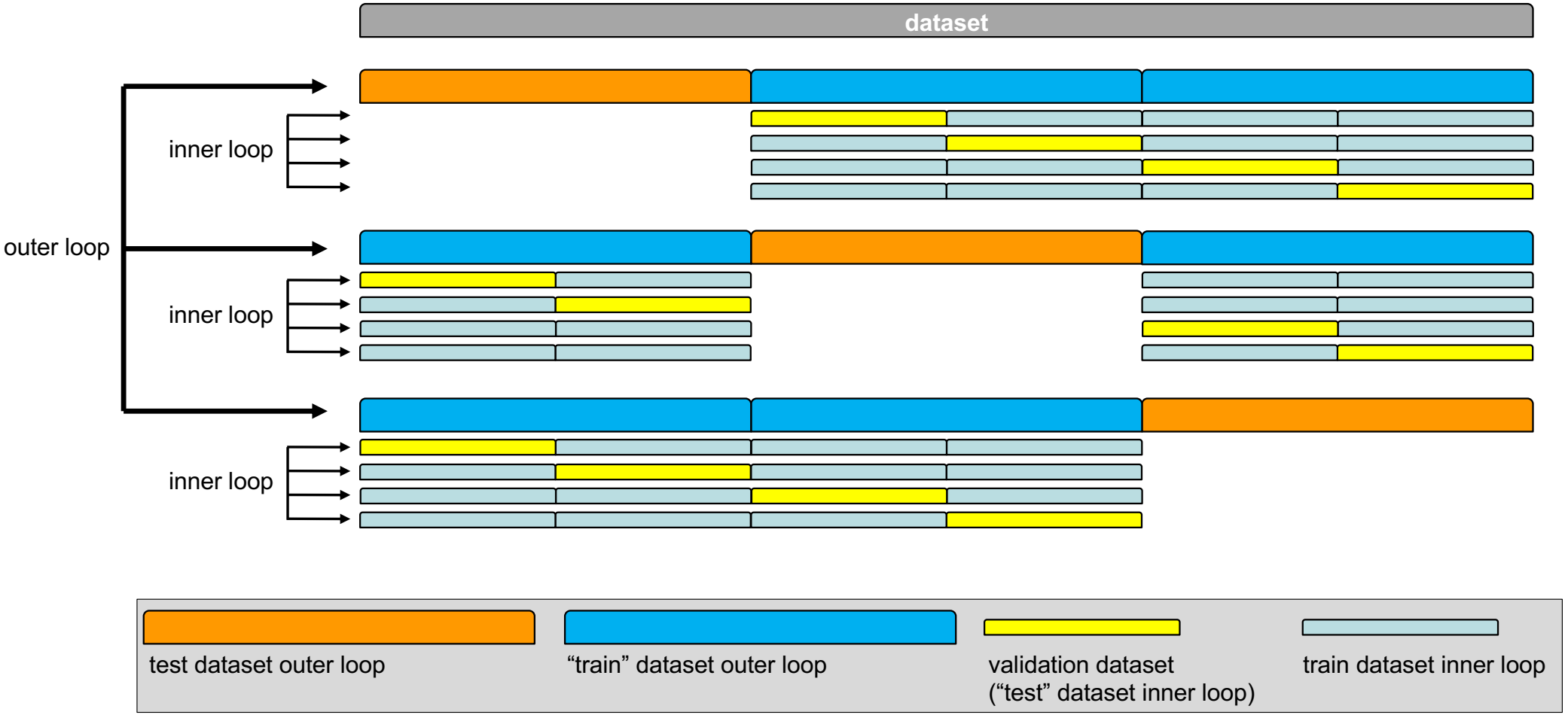
➔ Superior to nested-hold out

- Code: `sklearn.model_selection.GridSearchCV(…)`

Nested Cross-Validation

- Basic Idea: Nest two k-fold-cross validations inside each other
- Outer Loop – generates “outer train” and test dataset
  - Facilities model evaluation score as seen before
- Inner Loop – generates (real) train and validation from “outer train” dataset
  - Facilities hyperparameter tuning as seen before
- Outer and Inner loop may have different k’s (k_{outer}, k_{inner})
- Generalization of Nested-Hold-Out (nested-hold-out is the special case $k_{outer} = k_{inner} = 1$)

Nested Cross-Validation – Example for $k_{outer} = 3$ and $k_{inner} = 4$



Nested Cross-Validation - Discussion

- Advantages
  - Test dataset is never used to train/tune – no data leakage
    - Can compare (mean) score on train and test data to recognize overfitting
  - Easy to implement in scikit learn
  - Often good estimation of true score
  - Variance of both evaluation score and hyperparameters can be measured
    - Can recognize high variance of the model and hyperparameter values set by high variance of the scores

- Disadvantages
  - High computational cost: $O(k_{outer} * k_{inner} * h)$
  - Reduced size of training data, but less so than the other two methods
  - May have to use random sample to keep performance manageable

➔ Recommended (best practice) method for hyperparameter tuning

- Code: nest `sklearn.model_selection.GridSearchCV(…)` inside `sklearn.model_selection.cross_val_score(…)`

How to train our final model?

- Depending on the model evaluation method / the hyperparameter tuning method we may have trained multiple/very many models
- Which one do we deploy to production?
- None of them!

➔ We retrain the model (with the optimal hyperparameters we found) on the full dataset to get the final model

➔ How do we estimate the score of this model?

- If we already used cross-validation to estimate the score, we simply use this score
- If not, we should/may use cross-validation to estimate the score before the final training

Key Takeaways

- Classifier Scores
  - based on the „confusion matrix“: Accuracy, Precision and Recall, Sensitivity and Specificity, F-Score
  - based on the ROC-Curve: AUC
- Estimation of Classifier/Regressor Score
  - (Resubstitution), Hold-Out, Cross-Validation, Leave-One-Out, .632 Bootstrap

  never use general use more specialized
- Bias and Underfitting, Variance and Overfitting, Bias-Variance-Tradeoff
- Hyperparameter Tuning